Module 6 Critical Thinking - Option 2:

Adaptive Thresholding Scheme Accounting for Changes in Illumination

Alexander Ricciardi

Colorado State University Global

CSC515: Foundations of Computer Vision

Dr. Bingdong Li

October 25, 2026

Module 6 Critical Thinking - Option 2:

Adaptive Thresholding Scheme Accounting for Changes in Illumination

This paper explains how the Python program `Module6_CTA_Option2.py` examines two mugs under different lighting conditions using adaptive thresholding, a computer vision method that classifies pixels as black or white based on their brightness relative to a locally calculated threshold. One mug is glossy black, and the other is light-colored with carved flowers. Both sit on a light tabletop in front of a dark background. In the first photograph, the light comes from the white mug's side. It is then moved to the black mug's side for the second photograph. The illumination creates a problem for thresholding because the light reflecting on the mugs makes parts of the mugs' color brighter, and shadows create dark areas on the table and on the mugs themselves. This makes it difficult to separate the mugs from their surroundings based on pixel brightness alone. The program applies adaptive thresholding, then revises the adaptive settings to examine whether the results can be improved. This explanation examines the methods used, the resulting images, and the measurements to determine how well the program handles changes in illumination, including which boundaries remain visible and where the segmentation still has problems. Please see the Appendix section for the program's images and terminal outputs.

Preparing the Images

The first photograph is 2,134 × 1,765 pixels, while the second is 2,222 × 1,543 pixels. In `load_and_prepare()`, the program uses `cv2.imread()` to load each photograph, then `cv2.resize()` reduces both widths to 1,000 pixels while preserving their original proportions. This produces images of 1,000 × 827 and 1,000 × 694 pixels. Neither photograph is cropped before processing. This keeps both mugs, the tabletop, and the background in the images, including the shadows created by the different lighting directions. OpenCV reads the color

channels in blue, green, and red (BGR) order. The program uses `cv2.cvtColor()` to convert each photograph to grayscale, giving every pixel one brightness value between 0 (black) and 255 (white), instead of three color values. A 5 × 5 Gaussian blur is then applied using `cv2.GaussianBlur()` to reduce small variations in brightness before thresholding. However, the blur does not remove the larger shadows on the tabletop or the reflections on the black mug. These are part of the lighting conditions being examined, rather than simply random image noise. Both photographs go through the same preparation steps, and for each photograph, the initial and revised adaptive thresholding methods use the same blurred grayscale image.

Adaptive Thresholding

Adaptive thresholding calculates a separate cutoff for each pixel using the brightness values of its neighbors. This is useful when illumination varies across an image (Dey, 2025; OpenCV, n.d.). The program uses `cv2.ADAPTIVE_THRESH_GAUSSIAN_C`, which calculates a weighted local average and subtracts a constant, `C`. Both sets, the initial and revised adaptive thresholding, use `cv2.THRESH_BINARY_INV`, so pixels at or below their calculated threshold become white (255), while brighter pixels become black (0). However, a white pixel does not necessarily belong to a mug; see Figure 1. It only indicates that the pixel was selected by the thresholding method, which can also include shadows and parts of the background.

Reflections are not the only reason that parts of the mugs disappear. In a fairly uniform area, a pixel can have almost the same brightness as its neighbors. For example, suppose that a pixel and its weighted local average both have a value of 100. With `C=10`, its threshold is 90. Since 100 is greater than 90, inverse thresholding sets that pixel to black (0), even if it belongs to a mug. This example helps explain why much of a mug can remain black while its boundary appears white. A black area in the binary image therefore does not necessarily represent the

background (OpenCV, n.d.). The program initially uses a 31 × 31 window with `C=5`. Some of the mug boundaries become visible, although several areas remain thin or disconnected. After examining both photographs, the settings are changed to a 101 × 101 window and `C=10`. The larger window includes a wider area when calculating the local threshold, while increasing `C` reduces the number of pixels selected as white for a given local average. Both photographs are processed using these revised settings. Since the window size and `C` are changed together, their individual effects cannot be determined from this comparison alone. To examine the effects of these changes separately, an additional comparison used the same blurred grayscale images with two intermediate settings. With a 31 × 31 window and `C=10`, fewer details were selected on both mugs, but parts of their outlines also became harder to follow. With a 101 × 101 window and `C=5`, more of the rims and handles remained visible, along with more reflection details and background regions. These two intermediate settings were examined separately from the initial and revised results saved by the program.

The 101 × 101 window with `C=10` was retained as a compromise between these results. Several boundaries remain wider than in the initial images, while fewer local brightness changes are selected than with the larger window and `C=5`. This choice is based on the visible comparison of the two photographs. It does not establish that these settings are the best possible values or that the additional white pixels belong to the mugs. Code Excerpt 1 shows how the two adaptive results are calculated independently from the same blurred grayscale image.

Code Excerpt 1

Initial and Revised Gaussian Adaptive Thresholding

```
initial = cv2.adaptiveThreshold(
    blurred, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY_INV, INITIAL_BLOCK_SIZE, INITIAL_C,
)
```

```
    revised = cv2.adaptiveThreshold(
        blurred, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
        cv2.THRESH_BINARY_INV, REVISED_BLOCK_SIZE, REVISED_C,
    )
```

Note: Both adaptive calculations use the same blurred grayscale image. The initial adaptive settings are a 31 × 31 window and `C=5`, while the revised settings use a 101 × 101 window and `C=10`. The revised result is calculated from the original blurred input, not from the initial adaptive result.

The program saves the processed images, numerical measurements, and five Matplotlib figures in `mod_images/`. It also provides a five-step terminal explanation of the processing and results. All adaptive thresholding calculations are performed on the complete photographs. The revised adaptive image is the final result for each photograph.

Observed Results

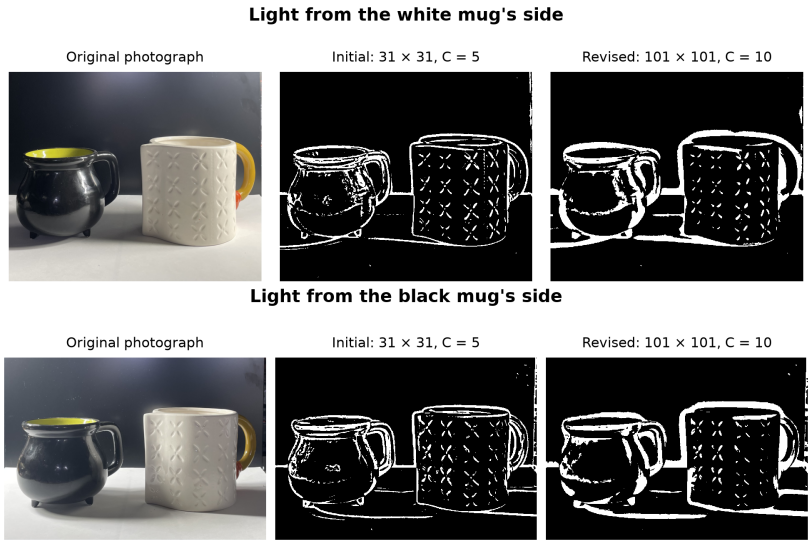
Figure 1 compares the two original photographs with the results obtained using the initial and revised adaptive thresholding settings. With the initial 31 × 31 window and `C=5`, thin white lines appear around parts of the mugs, particularly near the rims and handles. The raised flowers on the light-colored mug are also visible, although several areas of the outlines remain disconnected. The black mug presents a different problem because of its glossy surface. It reflects light and objects from its surroundings, producing bright areas within an otherwise dark object. Some of these reflections interrupt the white regions selected by the thresholding method. The light-colored mug also has areas where its outer boundary is difficult to distinguish from the tabletop, since their brightness values are close.

Changing the direction of illumination affects which parts of the mugs become visible in the binary images. Neither photograph produces a complete separation of the mugs from their surroundings. After revising the settings to a 101 × 101 window and `C=10`, several of the white

regions become wider and more continuous, making parts of the mug boundaries easier to follow. This is noticeable around some of the rims and handles. However, the revised settings also select more of the background, including shadows on the tabletop and areas near the dark backdrop. As shown in Figure 1, the improvement is not the same throughout the images.

Figure 1

Original Photographs and Adaptive Thresholding Under Both Lighting Directions



Note. The top row shows white-side illumination, and the bottom row shows black-side illumination. White (255) marks locally dark pixels, including mug features, reflection boundaries, and shadows.

The numerical measurements from `measure_masks()` help compare the initial and revised thresholding results. With illumination from the white mug's side, the initial adaptive image contains 66,762 white pixels. After changing the settings to a 101 × 101 window and `C=10`, the count increases to 96,910 pixels. A similar change occurs in the second photograph, where illumination comes from the black mug's side. The initial result contains 66,728 white pixels, increasing to 93,428 after the revision. These numbers show that the revised settings

select more pixels in both photographs, but the additional white regions are not necessarily part of the mugs. Some include shadows on the tabletop, darker areas of the background, and brightness changes produced by reflections. Therefore, an increase in white pixels does not necessarily indicate that the mugs have been separated more accurately from their surroundings.

Figure 2

Final Adaptive Thresholding Results Under Both Lighting Directions



Note. The top row shows white-side illumination, and the bottom row shows black-side illumination. Cyan marks the pixels selected by the revised adaptive thresholding settings, using a 101 × 101 window and `C=10`.

The program also counts connected white regions using `cv2.connectedComponents()` in `measure_masks()`. Pixels belong to the same region when they touch horizontally, vertically, or diagonally. For white-side illumination, the initial adaptive image contains 380 connected regions, compared with 150 in the revised adaptive image. With black-side illumination, the number decreases from 301 to 149. This is a noticeable reduction, but it does not necessarily

mean that the mug boundaries have become more accurate. Some of the smaller white regions may have disappeared, while others may have joined together. The same changes can occur in the background and shadows. Another consideration is that these measurements compare the initial and revised adaptive images, so the difference includes changes to both the window size and `C`. The program measures the number of white pixels and connected regions, but without reference masks identifying the actual mugs, it cannot determine how accurately the objects were segmented. Figure 2 compares the original photographs with the final adaptive images and their cyan overlays. The overlays make it easier to see which regions are selected, including parts of the mugs, shadows on the tabletop, and areas near the dark backdrop.

Comparison With the Research

Different methods can be used to compute the average brightness of pixels within local regions. For example, Bradley and Roth (2007) use an integral image, also called a summed-area table, which stores cumulative pixel brightness values to calculate local averages efficiently. However, the program uses `cv2.ADAPTIVE_THRESH_GAUSSIAN_C`, which takes a different approach. It calculates a weighted average of neighboring pixel brightness values, giving more weight to pixels closer to the center of the local region. The constant `C` is then subtracted from this average to calculate the local threshold. Although the calculations are different, both methods use local brightness information. This local brightness averaging is important for the two mug photographs because the illumination is not the same across the scene; the surroundings and each mug are affected differently by it. Moving the light from one side to the other changes the brightness of the mugs and their surroundings, creating reflections on the black mug and shadows on the tabletop. By calculating thresholds using nearby pixel brightness values, the program can account for some of these variations in illumination. This is the reason Bradley and

Roth (2007) use local brightness information in their approach. However, the results show that local thresholding has its limitations. The revised settings make parts of the mugs' boundaries easier to follow, but they also select additional shadows and background regions. This happens because the program evaluates differences in brightness without knowing whether those differences belong to a mug or something in its surroundings.

Sauvola and Pietikäinen (2000), in their research, examine how adaptive binarization of document images is affected by uneven illumination, noise, and differences in background brightness. They also use local image information to calculate thresholds, but their experiments involve documents, not photographs of objects reflecting light and casting shadows on other objects. For example, the black mug has a glossy surface that reflects light and objects from the surroundings, creating bright spots on the mug surface. Meanwhile, some of the shadows on the tabletop are darker than the neighboring pixels and may be selected by the inverse thresholding method. Note that the program only examines these differences in brightness; it cannot identify what produced them or the mugs themself. Nonetheless, Sauvola and Pietikäinen's (2000) research is relevant to this experiment because it examines how local thresholding can account for changes in illumination. However, the program’s results do not determine whether the pixels selected in the two mug photographs actually belong to the mugs or to their surroundings. To measure the accuracy of the segmentation, the program would need to compare its results with manually labeled reference masks identifying the mugs, which is out of the scope of this project.

Conclusion and Limitations

For the two photographs examined in this assignment, the revised adaptive thresholding settings produce more continuous regions around some of the mugs' boundaries, even after changing the direction of illumination. However, neither photograph produces a complete

separation of the mugs from their surroundings. The glossy black mug remains a problem because its reflections create bright areas that interrupt some of the selected regions. Parts of the light-colored mug are also difficult to distinguish from the tabletop, where their brightness values are similar. There is also information lost during image preparation. Converting the photographs to grayscale removes the color information, and thresholding reduces the different brightness values to black and white pixels. Once this information is removed, the binary image cannot restore it.

The numerical measurements also have limitations. The function `measure_masks()` counts white pixels and connected regions, but it cannot determine whether those pixels actually belong to either mug. Without manually labeled reference masks, the accuracy of the segmentation cannot be calculated. Another consideration is that the two original photographs have different dimensions and framing. Although both are resized to the same width, their heights remain different, and the scenes are not exactly the same. Therefore, differences between their measurements cannot be attributed to illumination alone. Both photographs were also examined when selecting the revised settings, meaning that the second photograph is not an independent test using an image that was never examined during development. The revised settings improve the visibility and continuity of several mug boundaries under both lighting conditions, but the experiment also shows the difficulty of separating objects when reflections, shadows, and background regions have similar brightness values.

References

Bradley, D., & Roth, G. (2007). Adaptive thresholding using the integral image. *Journal of Graphics Tools, 12*(2), 13–21. https://doi.org/10.1080/2151237X.2007.10129236

Dey, S. (2025). *Image processing and computer vision masterclass with Python: Learn advanced image processing techniques, computer vision fundamentals, and applications* (2nd ed.). BPB Publications.

OpenCV. (n.d.). *Image thresholding*. OpenCV 4.13.0 documentation. https://docs.opencv.org/4.13.0/d7/d4d/tutorial_py_thresholding.html

Sauvola, J., & Pietikäinen, M. (2000). Adaptive document image binarization. *Pattern Recognition, 33*(2), 225–236. https://doi.org/10.1016/S0031-3203(99)00055-2

Appendix

This section is reserved for screenshots from the program run, including the terminal explanation, original images, and the Matplotlib views.

Figure A1

Original Images

Light from the white mug side



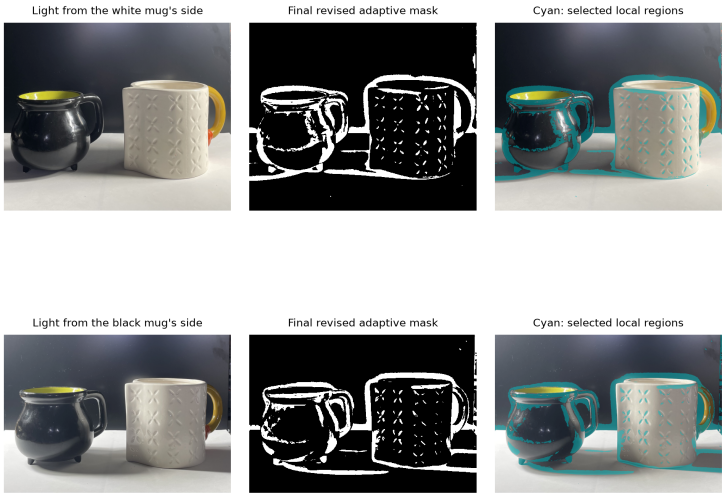
Light from the black mug side



Note: The images are photographs of a light source illuminating two mugs from the direction of the white mug and of a light source illuminating the mugs from the direction of the black mug.

Figure A2

Thresholding Operations Output Images



Note: The figure showcases the original images and Matplotlib output images illustrating thresholding Operations.

Figure A3

Terminal walkthrough

```
CSC515-CTAs — Python CTA_Module-6/Module6_CTA_Option2.py — 91×42

alex.omegapy@MacStudio CSC515-CTAs % .venv/bin/python CTA_Module-6/Module6_CTA_Option2.py
CSC515 Module 6 - Adaptive Thresholding Under Changing Illumination
All images, measurements, and explanations were saved before the guide.

================================================================================
STEP 1/5: Prepare the two full scenes
================================================================================

This assignment examines two mugs under different lighting conditions. One mug is glossy
black, and the other is light-colored with raised flowers. Both sit on a light tabletop
in front of a dark background. The first photograph is lit from the white mug's side. In
the second, the light comes from the black mug's side. The goal is to see whether the
same adaptive thresholding settings still work after that change.

Light from the white mug's side: original 2134 x 1765 pixels; processed 1000 x 827
pixels.

Light from the black mug's side: original 2222 x 1543 pixels; processed 1000 x 694
pixels.

In load_and_prepare(), cv2.resize() reduces both photographs to 1,000 pixels wide while
keeping their original proportions. Neither image is cropped, so the mugs, tabletop,
background, and shadows all remain. The first view shows the original photographs beside
their grayscale versions.

OpenCV reads color in BGR order, meaning blue, green, and red. Then cv2.cvtColor() gives
each pixel one brightness value instead of three color values. The grayscale image uses
uint8 values from 0, black, to 255, white. A 5 x 5 Gaussian blur reduces small
brightness variations before thresholding. With sigmaX=0, OpenCV calculates sigma from
the kernel size. Both adaptive calculations use this same blurred grayscale input for
each photograph.

The blur does not remove the larger shadows or the reflections on the black mug. Look at
its bright patches and the flowers on the white mug. These are details in the scene, and
they also change the brightness values. Treating all of them as random noise would miss
part of the problem.

This run uses Python 3.12.13; OpenCV 4.13.0; NumPy 2.5.3; Matplotlib 3.11.2.

Saved teaching view: figures/01_preparation.png

Enter C to continue or Q to quit, then press Enter:
```




```
Enter C to continue or Q to quit, then press Enter: c

================================================================================
STEP 2/5: Threshold the scene lit from the white mug's side
================================================================================

Start with the photograph lit from the white mug's side. The left panel shows the
blurred grayscale input, and the right panel shows the initial adaptive image. Look at
the rims, handles, and raised flowers to see which details remain visible.

Adaptive thresholding calculates a separate cutoff for each pixel using the brightness
of its neighbors. Here, ADAPTIVE_THRESH_GAUSSIAN_C calculates a weighted local average
and subtracts C. The initial settings are blockSize=31 and C=5, so the program examines
a 31 x 31 window and subtracts 5 from that average. THRESH_BINARY_INV makes pixels at or
below their threshold white (255), while brighter pixels become black (0).

A white pixel does not necessarily belong to a mug. It only means that the pixel met its
local threshold. For example, a shadow on the tabletop can be selected along with a dark
part of a handle. The table counts those selected pixels and their connected regions.

    Mask                White pixels    Image %    Regions
    initial_adaptive          66,762     8.0728        380

Some of the mug boundaries are visible, although several areas remain thin or
disconnected. The black mug's bright reflections interrupt parts of its outline. The
white mug has a different problem: parts of its boundary are close in brightness to the
tabletop.

Saved teaching view: figures/02_white_side_thresholding.png

Enter C to continue or Q to quit, then press Enter:
```




```
Enter C to continue or Q to quit, then press Enter: c

================================================================================
STEP 3/5: Use the same scheme with light from the black mug's side
================================================================================

Now look at the photograph lit from the black mug's side. The program uses the same
preparation and initial thresholding settings as in Step 2. Compare the right panel with
the previous result, particularly around the black mug's reflections and the tabletop
shadows.

The settings remain ADAPTIVE_THRESH_GAUSSIAN_C, THRESH_BINARY_INV, blockSize=31, and
C=5. Each pixel is still compared with its weighted local average minus 5. Values at or
below that cutoff become white (255); brighter values become black (0).

The black mug can still contain bright reflections, while the tabletop contains dark
shadows. This is why changing the light does not make every selected region part of a
mug. The table describes the pixels selected in this second photograph.

    Mask                White pixels    Image %    Regions
    initial_adaptive          66,728     9.6150        301

Changing the light affects which parts of the mugs appear in the binary image. The same
settings still pick up some boundaries, but they do not separate both mugs completely
from their surroundings. The next step revises the settings and checks both photographs
again.

Saved teaching view: figures/03_black_side_thresholding.png

Enter C to continue or Q to quit, then press Enter:
```



```
Enter C to continue or Q to quit, then press Enter: c

================================================================================
STEP 4/5: Revise the adaptive settings for both photographs
================================================================================

The initial results leave several boundaries thin or disconnected. For the revision, the
window increases to 101 x 101 and C changes to 10. Both photographs use these new
settings. The program starts again from each blurred grayscale image, so the revised
result is calculated independently of the initial binary image.

The larger window includes a wider area when calculating the local average. Increasing C
lowers the threshold, which selects fewer white pixels if that average stays the same.
However, the window also changes here, so its average can change too. This comparison
shows their combined effect; it cannot tell us how much each setting contributed.

Light from the white mug's side.
    Mask                White pixels    Image %    Regions
    initial_adaptive          66,762     8.0728        380
    revised_adaptive          96,910    11.7183        150

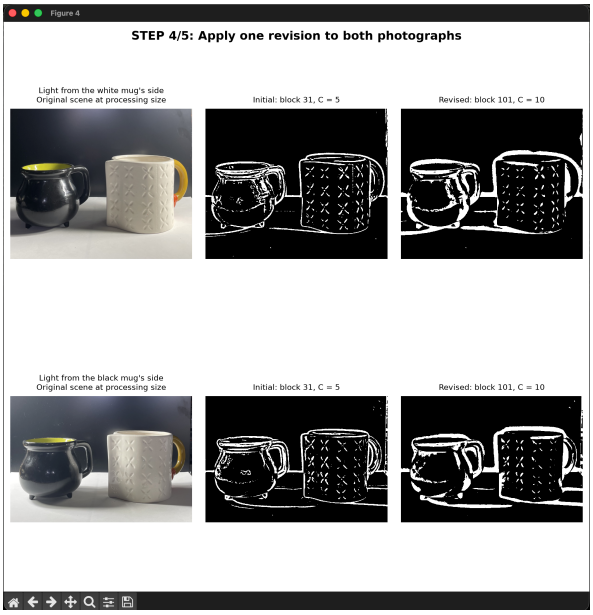
Light from the black mug's side.
    Mask                White pixels    Image %    Regions
    initial_adaptive          66,728     9.6150        301
    revised_adaptive          93,428    13.4622        149

Read across each row of the figure to compare the original photograph, initial result,
and revised result. Several white regions become wider and more continuous, making parts
of the rims and handles easier to follow. At the same time, more of the shadows and the
dark backdrop are selected. The improvement is not the same throughout the images.

The tables show more white pixels and fewer connected regions in both revised images.
Some smaller regions may have disappeared, while others may have joined together. Those
changes also occur in the background, so the numbers alone do not show whether the mugs
were separated more accurately.

Saved teaching view: figures/04_adaptive_revision.png

Enter C to continue or Q to quit, then press Enter:
```




```
Enter C to continue or Q to quit, then press Enter: c

================================================================================
STEP 5/5: Examine the results and their limits
================================================================================

The final comparison brings both lighting directions together. Each row shows the color
photograph, the revised adaptive image, and a cyan overlay. The revised adaptive image
is the final result. The overlay simply marks those same selected pixels on the
photograph.

Look at where the cyan areas fall. Some follow the mugs' rims and handles, but others
include shadows on the tabletop and bands near the dark backdrop. The boundaries are
easier to follow in some areas, although neither photograph gives a complete separation
of both mugs.

The black mug remains difficult because its glossy surface reflects light and objects
around it. Those bright patches still belong to the mug. Meanwhile, parts of the white
mug have brightness values close to the tabletop. Even the space inside a handle
contains background that can look similar to the handle itself. The raised flowers
create additional brightness changes. A local threshold cannot tell what produced all of
these differences.

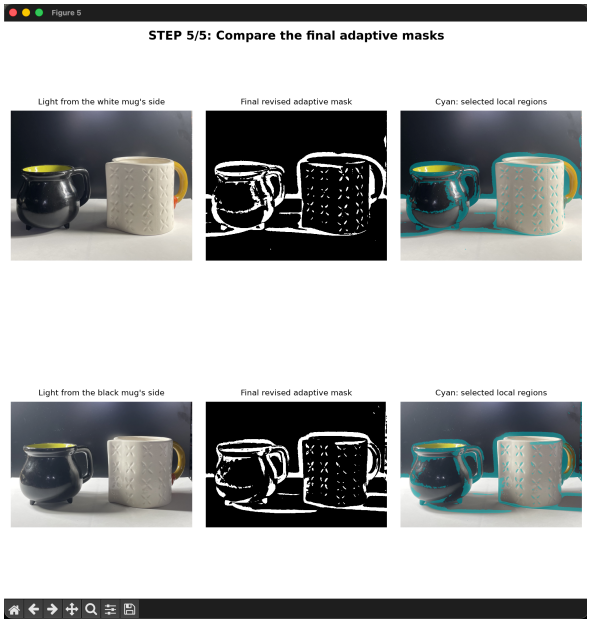
In measure_masks(), np.count_nonzero() counts the white pixels. Image % shows how much
of the full processed image they cover. Connected regions are groups of white pixels
that touch horizontally, vertically, or diagonally. These measurements help describe the
result, but they cannot tell whether the selected pixels actually belong to either mug.
Without manually labeled reference masks, segmentation accuracy cannot be calculated.

Another consideration is that the photographs have different dimensions and framing.
Resizing them to the same width does not make the scenes identical, so differences
between their measurements cannot be attributed to illumination alone. Both photographs
were also examined when choosing the revision. This shows how the settings work on these
two images; it does not establish how they will work on a new photograph.

For these photographs, the revised settings make several boundaries more continuous, but
they also include more background. That is the remaining problem with separating the
mugs using brightness alone.

Saved teaching view: figures/05_final_comparison.png

Enter Q to finish, then press Enter:
```




```
Enter Q to finish, then press Enter: q

Saved results: /Volumes/P-SSD/Omegapy/CSU-projects/CSC515-CTAs/CTA_Module-6/mod_images
alex.omegapy@MacStudio CSC515-CTAs %
```

Note: The figure showcases the terminal output and the images generated with the Matplotlib library.